

Department of Computer Science and Engineering

1. Subject Code: **SES-401** Course Title: **System Engineering**
2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks
3. **Prerequisites**
 - Programming in C; Programming Methodologies - I ; Computer Organisation and Architecture.
4. **Course Structure**
5 Units × 30 Hours (lectures, source-code reading, laboratory, PoW)

1. Course Description

This course introduces students to the internal architecture of modern Linux-based operating systems, bridging the gap between fundamental concepts and production-grade systems engineering. Through weekly hands-on laboratories involving kernel programming, scheduling, virtualisation, containers, and eBPF-based observability, students gain the practical skills required for backend infrastructure engineering, cloud computing, and high-performance systems development. The course follows a progressive “learn by building” approach: students transition from core OS internals to cloud-native Linux engineering, culminating in a complete systems portfolio. The curriculum emphasises hands-on production engineering and performance optimisation, deep dives into reading real Linux kernel source code, modern cloud-native infrastructure (containers, MicroVMs, eBPF), and an open-source contribution mindset with a debugging-first approach.

2. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Explain the architecture of modern operating systems including process management, memory hierarchy, kernel execution and system calls.
- CO2.** Design concurrent applications using processes, threads, synchronisation primitives and scalable scheduling techniques.
- CO3.** Develop high-performance Linux applications utilising modern filesystems, asynchronous I/O and networking interfaces.
- CO4.** Build secure execution environments using Linux namespaces, cgroups, virtualisation and container technologies.
- CO5.** Profile, debug and optimise production Linux systems using industry-standard observability and performance engineering tools.

3. Detailed Syllabus

Unit	Contents	Hrs
1	Operating System Foundations & Linux Architecture <i>Module A: Introduction to Operating Systems</i> Purpose and evolution of operating systems. Types of operating systems. Roles and responsibilities. Kernel, shell and user space. Resource management. The modern Linux ecosystem. <i>Module B: Computer Boot Process</i> Power-on sequence. BIOS and UEFI. Bootloaders (GRUB). Linux kernel loading. <code>initramfs</code>. System initialisation with <code>systemd</code>. Boot targets. <i>Module C: Processes and Threads</i> Program versus process. Process Control Block. Lifecycle and states. <code>fork()</code>, <code>exec()</code>, <code>wait()</code>. Zombie and orphan processes. Threads versus processes. Context switching. <i>Module D: System Calls & Linux Process Architecture</i> User versus kernel mode. CPU rings. Interrupts and traps. Common system calls. IPC overview. Process scheduler overview. The <code>/proc</code> and <code>/sys</code> filesystems. File descriptors. Process memory layout. Hands-on Laboratory & Proof-of-Work (PoW) • Linux terminal mastery, Git fundamentals, and process management. • Using <code>strace</code> and <code>ltrace</code> to inspect system and library calls. • PoW: Build a Mini Linux Shell supporting commands, redirection and pipes. • PoW: Implement a Process Explorer using the <code>/proc</code> filesystem.	30
2	Process Scheduling, Concurrency & Memory Management <i>Module A: CPU Scheduling Fundamentals</i> Scheduling objectives and metrics. Preemptive versus non-preemptive scheduling. Algorithms: FCFS, SJF, SRTF, Priority, Round Robin, MLFQ. Linux CFS and modern BPF-extensible scheduling (<code>sched_ext</code>). <i>Module B: Concurrency and Synchronisation</i> Race conditions, critical sections, mutual exclusion, deadlocks. Mutexes, semaphores, spinlocks. Atomic operations (CAS). Memory barriers. Lock-free concepts and RCU. <i>Module C: Virtual & Linux Memory Management</i> Logical versus physical memory. Paging, segmentation, demand paging. TLB, huge pages, swap space, the OOM killer. Kernel versus user memory. Buddy and Slab allocators. NUMA architecture. Hands-on Laboratory & Proof-of-Work (PoW) • Build multithreaded applications using POSIX Threads (pthreads). • Explore Linux scheduler behaviour using <code>perf</code> and <code>taskset</code>. • Analyse virtual memory using <code>vmstat</code>, <code>pmap</code> and <code>smem</code>. • PoW: Build a CPU Scheduling Simulator with performance visualisation. • PoW: Develop a multithreaded Thread Pool Library.	30

Unit	Contents (continued)	Hrs
3	File Systems, Storage Systems & High-Performance I/O <i>Module A: File System & Storage Fundamentals</i> Virtual File System (VFS). Inodes, superblocks, hard and symbolic links. Crash consistency. Block versus character devices. SSD, NVMe and DMA. <i>Module B: Modern Linux File Systems</i> ext4 architecture, XFS, Btrfs. Copy-on-Write. Journalling mechanisms. Page cache, buffer cache and Direct I/O. <i>Module C: High-Performance I/O & Networking</i> Blocking versus non-blocking I/O. Multiplexing with <code>select</code>, <code>poll</code>, <code>epoll</code>. Modern asynchronous programming with <code>io_uring</code>. Memory-mapped files (<code>mmap</code>). Zero-copy techniques (<code>sendfile</code>, <code>splice</code>). Kernel-bypass networking (DPDK, XDP). Hands-on Laboratory & Proof-of-Work (PoW) • Compare ext4 and XFS performance using <code>fio</code>. • Analyse storage performance using <code>iostat</code> and <code>blktrace</code>. • PoW: Implement a miniature in-memory filesystem. • PoW: Build a concurrent asynchronous TCP server using <code>epoll</code>, then upgrade it to <code>io_uring</code>.	30
4	Containers, Virtualisation & Cloud Operating Systems <i>Module A: OS Virtualisation & Resource Isolation</i> Linux namespaces (PID, mount, network, IPC, user, UTS). Control groups (cgroups v2) for CPU and memory isolation. Seccomp, AppArmor and SELinux overview. <i>Module B: Container Internals</i> OCI specifications. Docker architecture, <code>containerd</code>, <code>runc</code>. Image layers, union filesystems, OverlayFS. Container lifecycle and networking. <i>Module C: Hardware Virtualisation & Cloud-Native OS</i> Type-I and Type-II hypervisors (KVM, Intel VT-x). MicroVM architecture (Firecracker). Confidential computing. Edge computing fundamentals. Hands-on Laboratory & Proof-of-Work (PoW) • Build isolated execution environments using <code>unshare</code> and cgroups. • Analyse container layers using OverlayFS. • Build lightweight virtual machines using Firecracker. • PoW: Implement a lightweight container runtime in Go or Rust. • PoW: Create a secure root filesystem jail from scratch.	30

Unit	Contents (continued)	Hrs
5	Observability, Performance Engineering & Production <i>Module A: Performance Analysis & Kernel Tracing</i> Performance metrics, bottleneck identification, and benchmarking methodology. The Linux tracing ecosystem: perf, ftrace, kprobes, uprobes. Flame Graphs. <i>Module B: eBPF and Modern Kernel Instrumentation</i> The eBPF virtual machine, verifier, maps and helpers. XDP. BCC and libbpf. Writing user-space and kernel-space eBPF applications. <i>Module C: Kernel Debugging & Future OS Trends</i> Kernel panic analysis, core dumps, KASAN, KUnit. Distributed storage (Ceph). Real-Time Linux (PREEMPT_RT). The open-source patch workflow. Hands-on Laboratory & Proof-of-Work (PoW) • Generate Flame Graphs to identify CPU bottlenecks. • Trace filesystem and network activity using eBPF and libbpf. • Analyse kernel panic dumps and execute kernel selftests. • PoW: Develop an eBPF-based observability monitoring tool. • PoW: Perform a complete Root Cause Analysis for a kernel panic.	30
Total Instructional Effort		150

4. Recommended Resources

- Bovet & Cesati, *Understanding the Linux Kernel*; Love, *Linux Kernel Development*.
- Gregg, *Systems Performance* and *BPF Performance Tools*.
- Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (free online).
- Linux kernel source tree and **Documentation/** directory; man pages sections 2 and 7.
- Firecracker, containerd and libbpf official documentation.
- Linux System Programming Robert Love.